

Teljes együttes eloszlás tömör reprezentációja, Bayes hálók. Gépi tanulás: felügyelt tanulás problémája, döntési fák, naiv Bayes módszer, modellillesztés, mesterséges neuronhálók, k-legközelebbi szomszéd módszere

TELJES EGYÜTTES ELOSZLÁS TÖMÖR REPRESENTÁCIÓJA

- logikai igaz/hamis világgal és logikai következtetéssel vannak problémák:
 - ha nem teljes a tudás (tények vagy szabályok) akkor nem mindig tudunk logikai levezetéseket gyártani fontos kérdésekhez (döntésképtelenek leszünk)
 - ha heurisztikus (rávezető, tapasztalatra épülő) szabályokat vezetünk be, akkor a tapasztalat inkonzisztens (ellentmondó) lehet az elmélettel, tehát a logikai levezetés megint csak nem működik
- hiányos, részleges tudás kezelésére a logika nem optimális, ezért **valószínűségekk** kell dolgozni
- **a teljes együttes eloszlás tömör reprezentációja egy táblázattal történik**, ez az összes lehetséges elemi eseményt (minden lehetséges világállapotot) és ezek valószínűségét tartalmazza
 - n darab bináris változó esetén **a táblázat mérete 2^n** , tehát exponenciálisan nő
 - paraméterek száma: pl 3 logikai esetén $2*2*2 - 1 = 7$
 - a táblázatban szereplő valószínűségek összege pontosan 1
- problémája, hogy táblázat túl nagyra tud nőni pl $n = 30$ változó esetén már a gyakorlatban kezelhetetlen, több mint egymilliárd sor
- összes elemi esemény valószínűségét adja meg, ebből kiszámolható bármely kijelentés valószínűsége

Luk	Fogfájás	Beakad	$P()$
nem	nem	nem	0.576
nem	nem	igen	0.144
nem	igen	nem	0.064
nem	igen	igen	0.016
igen	nem	nem	0.008
igen	nem	igen	0.072
igen	igen	nem	0.012
igen	igen	igen	0.108

Valószínűség

- a gyakorlatban a tudás nem tökéletes, azaz vannak ismeretlen tények és szabályok
 - bizonyos dolgokról tudunk, bizonyosokról nem, ezeket akarjuk meghatározni
- véletlen hatásként kezeljük a tudás tökéletlenségét
 - fogfájás $\rightarrow$ luk = 0.8 (80%)
 - egy állítás/esemény valószínűsége változik annak függvényében, hogy mit tapasztalunk (megfigyelésekből, tényekből)
- priori - előzetes és posteriori - utólagos valószínűség

Véletlen változók

- egy véletlen változónak van egy neve és lehetséges értékei (értékkészlet, domainjei)
 - $A = D_A$
 - domain lehet logikai (igaz, hamis) és diszkrét (1, 2, 3, nap, felhő, hó) azaz megszámlálható, illetve folytonos (pl: $A < 3.2$)

Elemi esemény

- minden véletlen változóhoz értéket rendel
 - ha az $A_1, \dots, A_n$ véletlen változókat definiáltuk a $D_1, \dots, D_n$ domainekkel, akkor az elemi események a $D_1 \times \dots \times D_n$ halmaz
 - azaz az összes domain direkt szorzatával megkaphatjuk az elemi eseményeket
- például
 - két logikai véletlen változónk van: A és B
 - két domainjük D_A és D_B
 - ekkor az elemi események $D_A \times D_B$
 - A és B , $\neg A$ és B , A és $\neg B$, $\neg A$ és $\neg B$
 - mivel A és B domainjeibe 2, 2 tag van így $2 \cdot 2 = 4$ elemi esemény van
 - de ha lenne A , B és C , mely domainjeinek száma: 2, 3, 2 lenne akkor $2 \cdot 3 \cdot 2 = 12$ elemi esemény lenne

Valószínűség összesítő

- véletlen változók (A, B, X, Y) és változók értékei (a, b, x, y)
- $P(A = a)$, az a kijelentés valószínűsége
 - $P(\text{Idő} = \text{napos}) = 0.6$
- $P(A)$ a véletlen változó eloszlása
 - összes lehetséges értékére igaz lesz
- $P(A, B) = P(A \text{ és } B)$
- **$P(A = a \mid B = b) = \text{feltételes valószínűség}$**
 - azt jelenti, hogy mennyi a valószínűsége az a eseménynek, ha b -t tudjuk
 - $P(\text{luk} \mid \text{fogfájás}) = P(\text{luk} \wedge \text{fogfájás}) / P(\text{fogfájás})$
- Teljes együttes eloszlás táblázattal reprezentálható, ez tartalmazza minden eseményt és a hozzá tartozó valószínűségeket, a táblázat mérete 2^n , tehát nem skálázódik
 - tömbben is reprezentálható a függetlenség és feltételes függetlenség segítségével és úgy elviselhetőbb a méret
- **Függetlenség**
 - két véletlen változó akkor független ha az egyik sem tartalmaz információt a másikról
 - ha ismerem B értékét, abból semmit nem tudok meg A esélyeiről
 - a és b kijelentések akkor és csak akkor függetlenek ha $P(A=a, B=b) = P(A=a) \cdot P(B=b)$
 - A és B véletlen változók vagy változóhalmazok akkor és csak akkor függetlenek ha...
 - $P(A, B) = P(A) \cdot P(B)$
 - vagy, ha $P(A \mid B) = P(A)$
 $P(B \mid A) = P(B)$

- **Feltételes függetlenség**

- sima függetlenség ritka a gyakorlatban, inkább ez a jellemző
- gyakran előfordul, hogy két dolog nem független egymástól, de egy harmadik dolog ismeretében már függetlenné válnak
 - tipikusan A -nak és B -nek közös oka C
 - pl: fogfájás és a beakadás függetlenek, ha tudjuk, hogy van luk mert a luktól függnek, de ha nem tudjuk, hogy van luk akkor nem függetlenek
- a és b kijelentések feltételesen függetlenek c feltevésével akkor és csak akkor, ha $P(a, b | c) = P(a | c) * P(b | c)$, akkor a kijelentések nem feltétlenül függetlenek, hanem feltételesen függetlenek
 - tipikusan a -nak és b -nek közös oka c .
- A és B véletlen változók feltételesen függetlenek C feltevésével akkor és csak akkor, ha $P(A, B | C) = P(A | C) * P(B | C)$, vagy ekvivalensen

$$P(A, B | C) = P(A | C)$$

$$P(A, B | C) = P(B | C)$$
- erre épül a Bayes-háló, mivel a feltételes függetlenség segítségével az eloszlás reprezentációja tömöríthető

Az eloszlás reprezentációjának tömörítése a feltételes függetlenség segítségével: Tegyük fel, hogy a következő egyenlőség igaz:

$$P(A, B, C) = P(A, B|C)P(C) = P(A|C)P(B|C)P(C),$$

Ahol A , B és C jelöli változók diszjunkt (nincs közös elemük) halmazát. Itt az első egyenlőség a szorzat szabály, a második a egyenlőség fogalmazza meg a feltételes függetlenség feltevését. Ha A felvétele mellett teljesül, hogy $B_1, \dots, B_n$ kölcsönösen függetlenek.

BAYES HÁLÓK

- előnye, hogy a **feltételes függetlenséget kihasználva a véletlen változók teljes együttes eloszlását lehet Bayes hálóval reprezentálni kevesebb paraméterrel mint táblázatos formában**
 - n darab logikai változó esetén 2^n darab elemi esemény valószínűségét kellene megadni sima teljes együttes eloszlás esetén táblázatos formában, ehelyett a feltételes függetlenséget kihasználva a Bayes hálók a teljes együttes eloszlás reprezentációját tömörítik és így legfeljebb $n * 2^k$ paraméter kell (k = szülők maximális száma), itt a méret a változók számával lineárisan nő
 - **de a pontos teljes eloszlásra nem vezethetők vissza a bayes hálók**
 - változók sorrendje fontos
- **Bayes szabály** A Bayes szabály a és b kijelentésekre

$$P(a|b) = \frac{P(b|a)P(a)}{P(b)}$$

ami egyszerűen következik abból, hogy

$$P(a \wedge b) = P(a|b)P(b) = P(b|a)P(a)$$

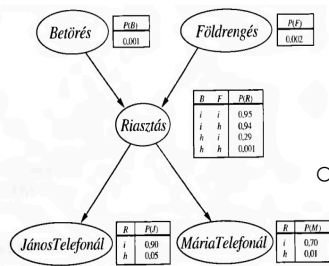
Általában is

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

ahol A és B változók vagy változók halmazai. A képletet valamely C feltételek mellett is lehet alkalmazni, azaz

$$P(A|B, C) = \frac{P(B|A, C)P(A|C)}{P(B|C)}$$

- a Bayes-háló egy irányított és körmentes gráf
 - a csúcsai a valószínűségi változók
 - irányított élek: Ha X-ből vezet Y-ba él akkor X az Y szülője és X nem független Y-tól



- de attól, hogy 2 változó nem független, az nem jelenti azt, hogy van közöttük él, de ha nincs él X és Y között akkor nincs közvetlen függőség sem közöttük
- minden X_i csúcsához tartozik egy $P(X_i | \text{Szülők}(X_i))$ feltételes valószínűségi eloszlás ami megadja a szülők hatását a csúcsonak megfeleltett változóra
 - lokális eloszlások
- minden változó feltételesen független a nem-leszármazottaitól, ha ismerjük a szüleit
 - emiatt a teljes együttes eloszlás végtelenül egyszerűen, a helyi feltételes valószínűségek szorzataként kapható meg: $P(X_1 \dots X_n) = \prod_{i=1}^n P(X_i | \text{Szülők}(X_i))$
- pl: Képzeld el, hogy a házadban van egy riasztó, ami betörésre vagy földrengésre is megszólalhat. Ha megszólal, két szomszédod, János és Mária felhívhat telefonon
 - János hívása függ a riasztástól, de ha tudjuk, hogy a riasztó megszólalt, János hívása már feltételesen független attól, hogy volt-e betörés, mert csak a riasztótól függ
 - $P(B, F, R, J, M) = P(B) * P(F) * P(R | B, F) * P(J | R) * P(M | R)$
 - láncszabály
- pl: Betegség diagnózis, a dátum befolyásolja az influenzát (téli gyakoribb), az Influenza pedig fejfájást okoz
 - a fejfájás függ az influenzától, de ha már tudjuk, hogy influenzások vagyunk akkor a fejfájás esélye független a dátumtól, mert csak az influenzától függ
 - háló: D (Dátum) → I (Influenza) → F (Fejfájás)
 - $P(D, I, F) = P(D) * P(I | D) * P(F | I)$
- sokszor a lekérdezés tünetekből kér következtetést az okozatra (pl: betegségekre). A “fordított” feltételes valószínűség az ami okozatból következtet tünetekre (ez sokkal időtállóbb, a szakértői tudás ilyen formájú)
 - a Bayes szabály visszavezeti a fordított képletre a lekérdezést
 - tehát van háttértudás ($P(\text{Fejfájás} | \text{Influenza})$) és aktuális tapasztalataink (Dátum = aktuális érték, Fejfájás = igaz és $P(\text{Influenza} | \text{dátum})$ közelítés) ebből következtethetünk a betegségekre a fejfájás tényéből az adott napon:

$$\begin{aligned}
 P(\text{Influenza} | \text{fejfájás}, \text{dátum}) &= \\
 &= \frac{P(\text{fejfájás} | \text{Influenza}, \text{dátum}) P(\text{Influenza} | \text{dátum})}{P(\text{fejfájás} | \text{dátum})} = \\
 &= \frac{P(\text{fejfájás} | \text{Influenza}) P(\text{Influenza} | \text{dátum})}{P(\text{fejfájás} | \text{dátum})}.
 \end{aligned}$$

Naiv Bayes algoritmus

- **valószínűségi alapú osztályozó** (Gépi tanulási algoritmus)
 - statisztikai következtetési módszer, amely az adatbázisban található példák alapján ismeretlen példákat osztályoz
 - (pl: email spam vagy nem spam, ekkor az adatbázis példa emaileket tartalmaz felcímkézve, hogy spam vagy nem spam)
- változók: döntési változó (A) + véletlen változók ($B_1, \dots, B_n$)
 - A lehet igaz (spam) vagy hamis (nem spam)
 - B_i logikai véletlen változó pedig az i -edik szó előfordulását jelöli (igaz, ha az email tartalmazza, hamis ha nem)
- feladata meghatározni, hogy a konkrét $b_1, \dots, b_n$ email esetén A mely értékére lesz a $P(A | b_1, \dots, b_n)$ feltételes valószínűség maximális

A klasszikus Bayes-tétel szerint annak a valószínűsége, hogy egy minta a C_k osztályba tartozik a megfigyelt $x = (x_1, \dots, x_n)$ tulajdonságok alapján:

$$P(C_k | x_1, \dots, x_n) = \frac{P(C_k) \cdot P(x_1, \dots, x_n | C_k)}{P(x_1, \dots, x_n)}$$

A fenti naiv feltételezést behelyettesítve a Naiv Bayes-osztályozó végleges képletét kapjuk:

$$P(C_k | x_1, \dots, x_n) = \frac{P(C_k) \cdot \prod_{i=1}^n P(x_i | C_k)}{P(x_1, \dots, x_n)}$$

Mivel a nevezőben lévő $P(x_1, \dots, x_n)$ minden osztály esetén ugyanaz az állandó (normalizáló tényező), az osztályozás során csak a számlálót maximalizáljuk:

$$\hat{y} = \arg \max_k P(C_k) \prod_{i=1}^n P(x_i | C_k)$$

- **Naiv Bayes feltevés egy közelítés**, mert nem tudjuk, hogy teljesül-e, az, hogy a feltételes függetlenségek fenn állnak-e, de bízunk benne, hogy elfogadható közelítést ad (ezért naiv a neve)

Bayes vs Naiv Bayes

- bemutató példa alapjai (az email spam vagy nem spam)
 - pl: két bemeneti változó a:
 - Szógyakoriság (mennyi alkalommal szerepel a “pénz” az emailbe)
 - Linktartalom (tartalmaz-e linket az email)
- Bayes esetén a **függőség figyelemben**
 - Bayes algoritmus nem tesz függetlenségi feltételezést a bemeneti változók között, az az figyelembe veszi az input változók közötti összefüggéseket
 - pl: a Bayes a fentebbi bemeneti változókból így jelezheti, hogy az email spam vagy sem:

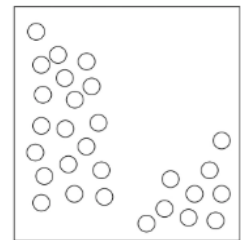
$$P(\text{spam} | \text{szógyakoriság}, \text{linktartalom}) = \frac{P(\text{szógyakoriság}, \text{linktartalom} | \text{spam}) \cdot P(\text{spam})}{P(\text{szógyakoriság}, \text{linktartalom})}$$

- $P(\text{spam})$ a spam valószínűsége az összes email között
- $P(\text{szógyakoriság}, \text{linktartalom} | \text{spam})$ a “pénz” szó gyakorisága és a linktartalom valószínűsége az összes spam email között
- $P(\text{szógyakoriság}, \text{linktartalom})$ a “pénz” szó gyakorisága és a linktartalom valószínűsége az összes email között
- bayes formulájában látszik, hogy a feltételes valószínűségek összetettebbek és tartalmazzák a változók közötti kapcsolatokat

- Naív Bayes esetén **naív függetlenség feltételezése**
 - a Naív Bayes algoritmus feltételezi, hogy a bemeneti változók függetlenek egymástól, ha ismert a kimenet, ezáltal egyszerűsíti a modellezését, de lehet, hogy nem teljesen reális a valósághoz képest
 - pl: fentebbi példából látható, hogy feltételezi, hogy a szógyakoriság és a linktartalom nem függenek egymástól, ha tudjuk, hogy az email spam vagy nem spam, a Naiv Bayes algoritmus formulája:

$$P(\text{spam} | \text{szógyakoriság}, \text{linktartalom}) = P(\text{szógyakoriság} | \text{spam}) \cdot P(\text{linktartalom} | \text{spam}) \cdot P(\text{spam})$$

- $P(\text{szógyakoriság} | \text{spam})$ a "pénz" szó gyakoriságának valószínűsége az összes spam email között
- $P(\text{linktartalom} | \text{spam})$ a linktartalom valószínűsége az összes spam email között
- $P(\text{spam})$ a spam valószínűsége az összes email között
- A függetlenségi feltételezés világosan látható a Naív Bayes formulájában, ahol az egyes változók valószínűségei szorzat formájában számíthatók ki, anélkül hogy figyelembe vennék a közöttük lévő lehetséges összefüggéseket



felügyelet nélküli tanulás

GÉPI TANULÁS

Felügyeletlen tanulás

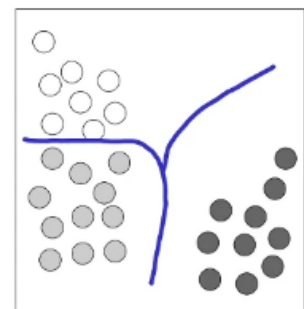
- címke nélküli tanítópéldák
 - csak $x_1, \dots, x_n$ alakban adottak a példák (x_i eleme X -nek)
- nem függvényt kell keresni, hanem mintázatokat, összefüggéseket kell keresni
 - kicsit olyan, mintha kapnál egy hatalmas doboz építőkockát leírás nélkül: a feladatod nem az, hogy lemásolj egy kész házat, hanem az, hogy csoportosítsd őket szín, méret vagy forma szerint, és észrevedd a belső logikát
 - klaszterezés, anomália detektálás

Megerősítéssel tanulás

- $X \rightarrow Y$ függvény létezik, de a tanítópéldák direkt módon nem adottak
 - X : állapotok, Y : cselekvések
- itt a feladat egy optimális startégia (maximalizálás) megtanulása az állapottérben, próba-szerencse alapon, jutalmak és büntetések útján tanul (pl: kiskutya)

Felügyelt (induktív) tanulás

- címkézett tanítópéldák, tipikusan osztályozó feladatok (pl: karakter felismerés)
- adott egy $f: X \rightarrow Y$ függvény és $(x_1, f(x_1)), \dots, (x_n, f(x_n))$ alakú tanító példák, ahol x_i eleme X -nek
 - spam szűrő példa
 - X : értékek halmaza (pl: emailek halmaza)
 - Y : osztály címkék (pl: {spam, nem spam})
 - a tanítópéldák a kézzel osztályozott emailek

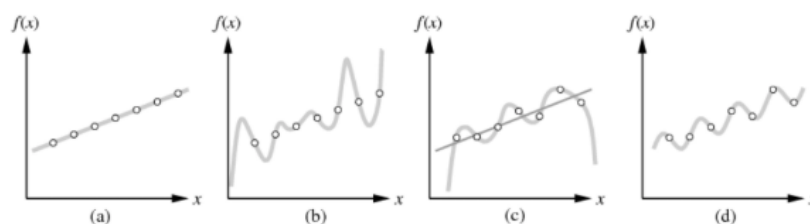


felügyelt tanulás

- cél keresni egy f -re illeszkedő $h: X \rightarrow Y$ hipotézis függvényt a példákban rejlő minták alapján, tehát az **általánosítás** a feladat
 - mintákat jellemzőkkel (feature) írjuk le, ezek lehetnek binárisak (igen/nem), véges diszkrét értékek vagy valós értékek
 - azaz egy függvényt amely illeszkedik az adott tanítópéldákra egy H hipotézis térben, más szóval, „a függvényt mindig adott alakban (adott fokszámú polinom, stb.) keressük”
 - ez egy függvény közelítés, gyakorlatban nem kell pontosan illeszkednie feltétlenül, elég ha közel van
 - h függvény konzisztens az adatokkal, azaz $h(x_i) = f(x_i)$ minden tanító példára
 - tanulás realizálható, ha létezik h eleme H , amelyre h konzisztens

- **Modellillesztés**

- Hipotézistér (H)
 - az összes olyan függvény halmaza, amelyet a tanuló algoritmus egyáltalán képes „kigondolni” vagy felvenni (pl. minden lehetséges döntési fa vagy minden lehetséges függvény)
- adottak a pozitív és negatív példák, ahol minden változó értékét megadjuk, a példa címkéjével együtt
- az a folyamat amikor a hipotézistérből (H) megkeressük azt a függvényt (h), amely a legjobban illeszkedik a tanító ad adatainkra (minimalizálja hibát)



(a) és (b) ábra: az általánosabb (azaz bővebb) H lehet akár a rosszabb választás is. (c) és (d) ábra: az is fontos hogy H tartalmazza a megfelelő hipotéziseket (pl. akármilyen általános polinom osztály nem lesz periódikus soha valós számok felett).

- Típusai a kimenete (Y , osztálycímkek) szerint:
 - **Osztályozás:** ha a kimenete egy diszkrét osztálycímke igen/nem, alma/körte)
 - **Regresszió:** ha a kimenete egyfolytonos számérték (pl: hőmérséklet)
- indukció problémája
 - f jó közelítése olyan amely a példákon kívül is jól közelíti f -et, azaz jól általánosít
 - ha nem általánosít jól akkor **magolásról** van szó
 - pl: az a h amelyre $h(x) = f(x)$ minden példára, egyébként $h(x) = 0$, azaz tökéletesen tanulja a példákat, de a lehető legrosszabban általánosít (általában)

- magolás ellen a tömör reprezentációra (kevés paraméter legyen) kell törekedni
 - Occam borotvájaa elv (legtömörebb modell választása ha más alapján nem tuduk dönteni)
 - de azért elég kifejezőnek kell lennie a reprezentációnak, hogy a tanító példákat elég jó közelíteni tudják
 - alulillesztés
 - a modell túl egyszerű ahhoz, hogy megragadja az adatokban lévő összefüggéseket (pl: egyenest akarunk egy hullámzó ponthalmazra illeszteni)
 - ilyenkor még a tanító adatokon is rosszul teljesít
 - túlillesztés
 - a magolás ennek egy speciális esete
 - a lényege, hogy betanulja a tanító adatok, ezzel a zajt is, nem csak a valós összefüggéseket, ami miatt csökken az általánosító képesség
 - ekkor a tanító halmazon a hiba nagyon kicsi, de a teszhalmazon már nagy
 - akkor fordul elő, ha túl kevés a tanító példa vagy túl általános a modellünk
- **priori ismertek**
 - ha nincs előismeretünk (**priori ismeret**) akkor nem tudunk a magolásnál jobb módon tanulni
 - lényege, hogy a H halmaz és az algoritmus maga a priori ismeretek alapján kerül megtervezésre
- **kiértékelése**
 - példákat két részre kell osztani: tanító halmaz (90% a példáknek) és teszt halmaz (10%-a a példáknek)
 - de $\frac{2}{3}$ és $\frac{1}{3}$ arány is jó
 - pontosság: ha a tanító példákon nő is, az nem jelenti, hogy teszt halmazon is fog
 - javítási lehetőségek
 - bootstrap módszer
 - véletlen mintavételezés visszatevéses módszerrel
 - N elemű teljes mintahalmazból válasszunk ki visszatevéssel N elemet, ez lesz a tanítóhalmaz
 - a maradék pedig a teszhalmaz
 - annak esélye, hogy egy elemet nem választunk be a tanítóhalmazba 36.8 %, tehát ennyi marad ki és lesz a teszhalmazba

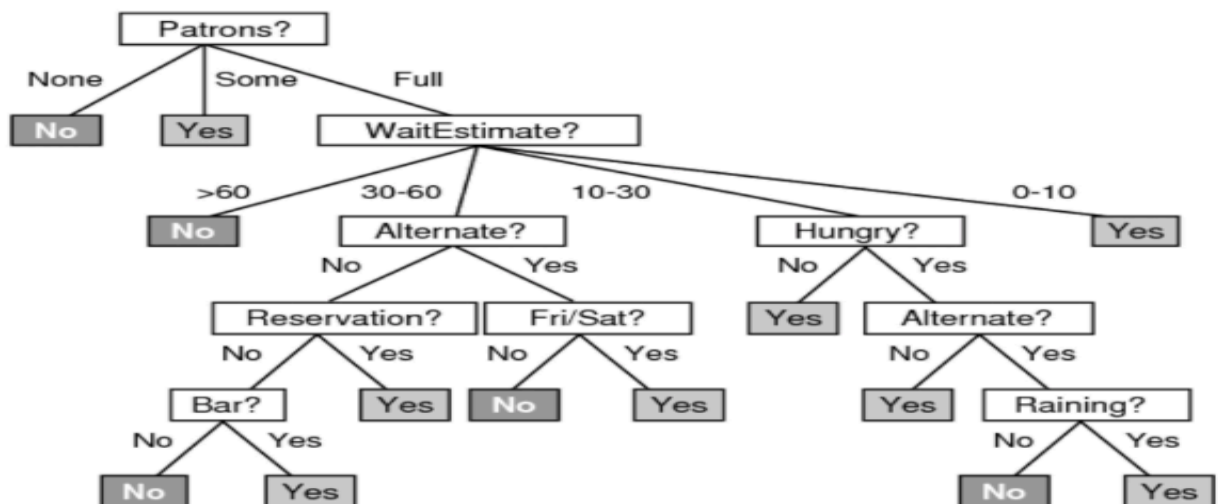
- kereszt validáció
 - példák felosztása N egyenlő részre (akkor jó ha kevés van, általában 10)
 - $N-1$ részen tanítunk és egyen halmazon validálunk
 - ezt N alkalommal ismételjük, mindig másik halmazt kihagyva a validálásra
 - így N *darab* tanítást és tesztelést kell elvégezni és ezek átlagos eredményével jellemezzük a módszert
 - speciális esete a leave-one-out
 - csak 1 elemű a teszhalmaz
- kukucskálás, azaz peeking
 - tanító halmaz (70%), teszhalmaz (15%) és validációs halmaz (15%)
 - validációs halmaz a fejlesztés / finomhangolásra (hiperparaméterek beállítása, túlillesztés kiszűrése)
 - teszt halmaz pedig a végső ellenőrzésre (a tanítás során a modell ezekkel nem érintkezik)
- hibák mérése
 - X osztálycímkéjével egyezik az output értéke, akkor a hiba 0, különben 1
 - Hibafüggvények
 - ezek az elvárt és kapott outputok értéke közti eltérést mérik ($t - o$)
 - négyzetes hiba
 - átlagos négyzetes hiba (MSE)
 - a target és output vektorokról semmit nem feltételez, ezért általános feladatokra is jó
 - 0-t csak akkor ad ha a target (t) és output (o) vektorok tökéletesen egyeznek
 - középhiba (RMSE)
 - kereszt entrópia (CE)
 - diszkrét eloszlások eltérésének mérésére szolgál
 - osztályozási feladatnál ha a háló kimeneteit úgy értelmezzük mint valószínűségi becslés akkor ezt érdemes
- tanítási tippek és trükkök
 - adatok normalizálása (standardizálás)
 - egyes jellemzők értéktartományát egységes skálára hozni
 - pl: $\min = -1$, $\max = 1$
 - adatok randomizálása
 - sokat segíthet ha az adat vektorokat véletlenszerű sorrendbe rakjuk (semi-batch)
 - minden epoch (tanítási folyamat) után érdemes randomizálni az adatokat
 - súlyok inicializálása
 - kis random értékkel inicializáljuk, pl: $[-0.1, 0.1]$ közötti véletlen érték
 - jó inicializálás segít a hiba visszatérjesztésébe

- learning rate hangolás
 - egy alapvető hiperparaméter a gépi tanulásban és az mi-ben
 - azt határozza meg, hogy a modell mekkora lépésekkel módosítsa a belső paramétereit az optimalizációs folyamat során
 - túl nagy: nincs tanulás és ugrál a hiba felületen
 - túl kicsi: lassú tanulás, könnyebben elakad a lokális optimumon
 - általános heurisztika rá, hogy olyan nagy learning rate-tel indítunk amivel még van tanulás, és mikor a hiba már nem csökken akkor elkezdjük csökkenteni a learning rate-t (felezgetés)

Döntési fák

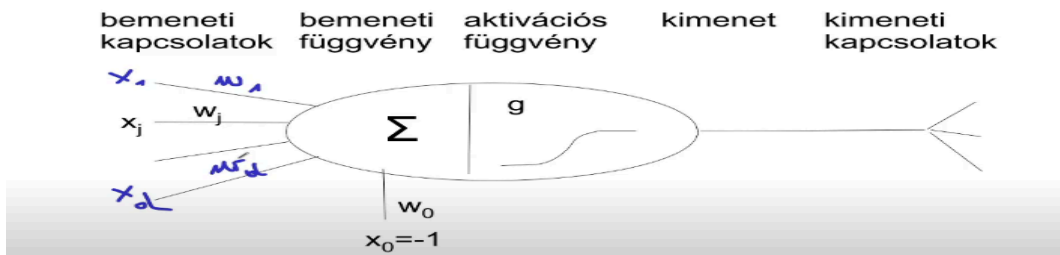
- felügyelt tanulás egy konkrét példája, algoritmus
- előnye, hogy a döntései könnyen magyarázhatóak, mert emberileg is értelmezhető lépésekben jutunk el a döntésig (mesterséges neuron hálókra ez nem igaz pl)
- egyes példákban hiányozhatnak bizonyos attribútumok
 - megoldás új példák felvétele ahol ezek már ott vannak
- hierarchikus fa struktúrája:
 - **gyökér:** a legelső kérdés, ahol a döntési folyamat indul
 - **belső csomópontok:** tesztek egy-egy jellemzőre (pl: “vannak vendégek”, “kell-e várni”)
 - **élek:** tesztek lehetséges kimenetei (igen/nem, vagy más csomópont, azaz új teszt)
 - **levelek:** végső döntések, vagy osztálycímkek, azaz az eredmény
- fa felépítése
 - cél, hogy minnél kevesebb kérdéssel, azaz a lehető legkisebb mélységgel építsük a fát
 - ez egy mohó algoritmus, minden lépésben azt választjuk, mely a legjobban szeparálja a pozitív és negatív példákat, azaz a legtöbb információt hordozza
 - a gyökér az lesz amelyik a legjobban szeparálja a példákat
 - ahhoz, hogy eldöntsük melyik a legjobb mérni kell a bizonytalanságot
 - **entrópia:** azt méri, hogy mennyire kevert vagy zajos egy adathalmaz, ha például minden példa ugyanabba az osztályba tartozik akkor az entrópia értéke 0 (tiszt), akkor 1 az értéke (maximális bizonytalanság) ha az osztályok aránya fele-fele
 - **információnyereség (gain):** megmutatja, hogy mennyit csökken az entrópia, ha egy adott jellemző szerint kettévágjuk az adatot, ezért mindig azt a jellemzőt választjuk amelyiknél ez az érték a nagyobb
 - speciális esetekben megállítjuk a rekurziót
 - ha csak pozitív vagy negatív példa van, akkor levélhez értünk, megcímkézzük megfelelő módon
 - ha üres halmazról van szó: alapértelmezett érték, pl. a szülőben a többségi döntés
 - ha pozitív és negatív példa is van, de nincs több változó: ekkor a többségi szavazattal címkézhetjük a levelet

- túlilleszkedés elleni védekezés
 - ha hagyjuk, hogy a fa addig nőjön még minden levélbe csak 1-1 példa marad, akkor a fa magolja az adatokat, új adatokon meg rosszul fog futni
 - megoldások erre:
 - **nyesés**
 - előnyesés (korai megállás)
 - még a fa építése közben megáll, ha teljesülnek ezek a feltételek:
 - a fa elérte a max mélységet
 - egy csomópontban lévő példák száma egy bizonyos küszöb alá esik
 - információnyereség túl kicsi
 - utó-nyesés
 - hagyjuk a fát először megnőni, majd lentről felfele haladva kezdjük levágni azon ágakat (élek), melyek nem javítják érdemben a modell pontosságát, egy különálló validációs halmazon
 - **minimum példaszám a levelekbe**
 - előírhatjuk, hogy egy levél csak akkor jöhet létre, ha legalább N számú példát tartalmaz
 - megakadályozza, hogy a fa egyedi esetekre, azaz zajra hozzon létre új ágot, élt
- példa feladat:
 - *leírása:* döntünk el, hogy érdemes-e asztalra várni egy étteremben. Ez egy 2 osztályos (igen, nem) osztályozási feladat, azaz döntési feladat. Az X halmaz a változók (Vendég, Éhes stb.) értékeinek a vektora, minden változó diszkrét.

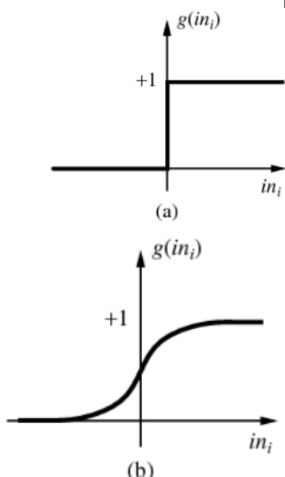


MESTERSÉGES NEURONHÁLÓK

Különálló neuron



- inputok
 - egy input áll egy értékből (x_i) és egy súlyból (w_i)
 - érték általában tetszőleges valós szám (néhány esetben lehetnek korlátosak)
 - súlyok valós számok, tanítás során ezeket módosítja az algoritmus
 - bias input, azaz eltolás súly
 - értéke ($x_0 = -1$), míg súlya w_0
 - fix bemenet, mindig -1
 - szerepe, hogy a függvény mindig w_0 -nál váltson
- belső felépítése:
 - **bemeneti függvény**
 - inputok összege + bias input
 - $\text{input} = \text{input_érték} * \text{input_súly}$
 - belső szorzattal írható le
 - **aktivációs függvény**
 - $g()$ a jelölésű, **nemlineáris** függvény
 - nemlineáris fontos különben tök felesleges lenne, mert súlyokkal is helyettesíthető volna
 - a bemeneti függvény értékén fut le, célja ha jó a bement akkor 1-et vagy ahozz közeli értéket adjon, ha pedig rossz akkor 0-t vagy 0-hoz közeli értéket adjon
 - manapság már nem követelmény, hogy $[0, 1]$ intervallumból adja az eredményét (ReLU)
 - w_0 -nál azaz eltolás súlynál váltanak
 - több implementációja van van
 - **Küszöbfüggvény** (perceptron neuron)
 - $g(x) = 0$ ha $x \leq 0$, $g(x) = 1$ ha $x > 0$
 - 0 vagy 1 érték
 - gradiens módszer nem alkalmazható vele
 - **Sigmoid függvény**
 - $g(x) = 1 / (1 + e^{-x})$
 - $(0, 1]$ között, azaz tekinthető úgy, mint a küszöbfüggvény folytonos közelítése
 - matematikailag könnyebben kezelhető (deriválható pl)



- **Softmax függvény**

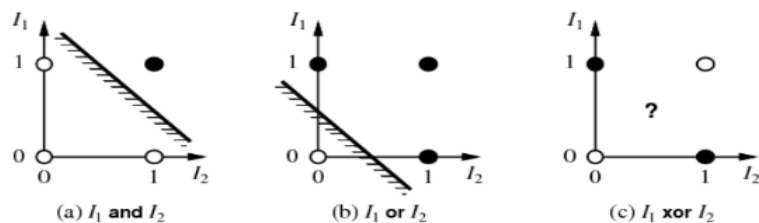
- ha a neuronok kimeneteit egyes osztályok valószínűségeként akarjuk akkor ezt használjuk
- hasonló mint a Sigmoid, de itt már a valószínűségek összege 1

- **Rectifier aktiváció**

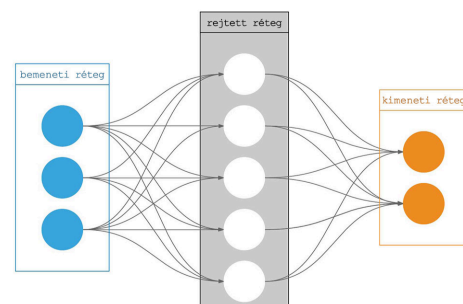
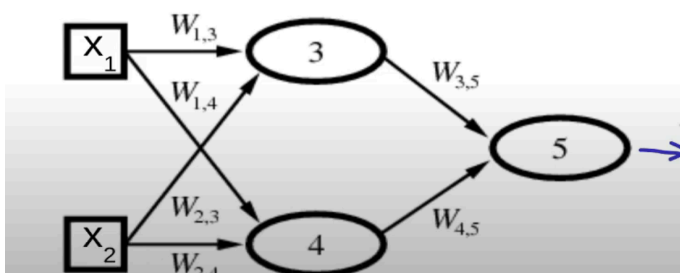
- $g(x) = \max(0, x)$
- ha a neuron ezt használja akkor ReLU neuronnak hívjuk
- ha negatív akkor 0, különben marad az érték

- kimenete:

- aktivációs függvény adja
- $h(x) = g(w * x)$
 - nem sima szorás hanem belső szorzat
 - $d - 1$ dimenziós hipersíkot határoz meg, ami nem feltétlen megy át az origón (eltolássúly miatt)
- 2 dimenzió input ($d = 2$) esetén egy egyenes lesz a hipersík
 - dimenzió az, hogy hány értékből áll pl (felhős idő van, esik az eső)
 - a hipersíkkal 2 részre osztja a teret és $w \cdot x > 0$ esetén (sigmoidot használva $g(w \cdot x) > 0.5$) elfogadja az inputot, egyébként nem
- egy neuronnal hiba nélkül csak lineárisan szeparálható függvényeket lehet reprezentálni (pl: and, or , de a xor-t már nem lehet)



Többrétegű neuron hálók



- olyan mesterséges intelligencia modellek amelyek egymásra épülő neuron rétegekből állnak
- előrecsatolt hálózat
 - nincs benne kör
 - az információ az első, bemeneti rétegtől halad előre felé a hálózatba a rejtett rétegeken keresztül egészen a kimeneti rétegig

- rétegek
 - input (bementi) réteg
 - rejtett rétegek
 - összetett összefüggések modellezésében segítenek ezek a rétegek
 - egy darab rejtett réteggel már bármilyen folytonos függvény közelíthető (konvex tér), de minnél több van annál komplexebb
 - 2 darabbal elvileg már minden megoldható (hagyományos neuronháló)
 - minnél több neuronból áll annál pontosabb a közelítés
 - output (kimeneti) réteg(ek)
- nincs visszafelé kapcsolás és a rétegeken belül se
- tanítása optimalizálás alapú (mint eddig)
 - differenciálhatónak kell lennie (gradiens módszer)
 - veszteségfüggvény = negatív log likelihood
- backpropagation algoritmussal tanul, ez a gradiens számításához van
 - hátulról visszafelé hibafejtés, célja, hogy a kimeneti hiba alapján módosítsa a súlyokat
- rekurrens hálók
 - körök engedélyezettek
 - dinamikus a rendszer (vagy konvergál vagy nem)

Modell tanítása

- a tanítás egy iteratív folyamat
1. Előreccsatolás: Átküldjük az adatot a hálón és kapunk egy jóslatot
 - a. pl: beadunk egy "a" karakterről képet és a háló a véletlen súlyai miatt az mondja, hogy 30% eséllyel egy "a" karakterről van szó és 70% az esély, hogy nem
 2. Hibaszámítás: Megnézzük, mennyire tért el a jóslat a valóságtól (hiba számítás, veszteség függvény)
 - a. pl: a háló 0,3-at (30%-ot) mondott de a helyes az 1.0-a lenne akkor nagy a hiba (veszteség), tehát rossz tipp volt
 3. Hibavisszaterjesztés (backpropagation): A kimenettől visszafelé haladva kiszámoljuk, hogy melyik súly mennyivel felelős a hibáért
 - a. pl: *"az utolsó előtti rétegben ez a súly túl nagy volt, az a másik meg túl kicsi, ezért hittük azt, hogy ez nem egy "a" karakter"*
 - b. gyakorlatban a backpropagation problémás, mert a mélyebben lévő rétegek nem tanulnak
 4. Súlyfrissítés: Úgy módosítjuk a súlyokat, hogy a hiba csökkenjen
 - a. pl: a súlyokat úgy állítjuk át, hogy legközelebb, ha ugyanezt a képet látja, már ne 0.3-at, hanem mondjuk 0.35-öt mondjon

Minták közötti távolság mérés

- ha a minták n jellemzővel vannak reprezentálva a következő módokon tudunk távolságot mérni a minták között:

- Euklideszi távolság: $d(i, j) = \sqrt{\sum_k (x_i[k] - x_j[k])^2}$, $k = 1, \dots, n$

- Manhattan távolság: $d(i, j) = \sum_k |x_i[k] - x_j[k]|$, $k = 1, \dots, n$

- Cosinus távolság: $d(i, j) = \frac{x_i \times x_j}{||x_i|| \cdot ||x_j||}$

A k-NN (k - Nearest Neighbour) osztályzó

- egy osztályzó algoritmus (felügyelt tanulás), mely a minták közötti távolság alapján osztályoz új mintákat úgy, hogy nem épít modellt a tanulási fázisban, csak eltárolja a tanító adatokat
- szüksége van hozzá egy tanító adatkészletre, mely a mintákat és a hozzájuk tartozó osztály címkéket tartalmazza
- menete
 - ha új mintát osztályozunk akkor meg kell határozni a k legközelebbi ismert mintát a tanító adatkészleten a távolság alapján, ezután az új minta osztály címkéjét a k legközelebbi minta alapján határozzuk meg
 - **választás osztályozásnál**
 - többségi szavazás
 - leggyakorabbi megközelítés, ekkor az új minta osztálycímkéje az a címke lesz amelyik a k legközelebbi mintába a leggyakoribb
 - Például, ha $k = 3$, és a k legközelebbi minta 2 "A" osztályú mintát és 1 "B" osztályú mintát tartalmaz, akkor az új mintát "A" osztályba soroljuk be
 - súlyozott szavazás
 - ekkor az új minta szomszédjainak szavazatait a távolság reciprokjával súlyozzuk, minnél közelebbi a szomszéd annál nagyobb súllyal súlyozzuk
 - akkor hasznos, ha a k legközelebbi minta nem egyenletesen oszlik el
 - **választás regresszióánál**
 - ilyenkor a k legközelebbi szomszédjainak kimeneti értékeinek átlaga
 - **1-NN egy speciális esete**
 - $k = 1$, az az új minta osztálycímkéjét a legközelebbi ismert minta osztálycímkéje határozza meg
 - egyszerűbb, de hajlamosabb zajra és az outlierekre
- előnyei
 - egyszerű implementáció és érteni is egyszerű
 - kis adatbázisok esetén hatékony
 - nem feltétlenül feltételezi a lineárisan szeparálható osztályokat

- hátrányai
 - adatbázis méret alapján romlik a teljesítmény
 - érzékeny a zajra és az outlierekre
 - k paraméter kiválasztása is befolyásolja a teljesítményt (érdemes páratlant választani, hogy elkerüljük a döntetleneket)
 - túl kicsi akkor érzékeny a zajra és a kiugró adatokra (túlillesztés)
 - túl nagy akkor a döntési határok elmosódnak, és a ritkább osztályok teljesen eltűnnek a többség mellett (alulillesztés)
- példa
 - Tegyük fel, hogy van egy tanító adatkészletünk, amely virágokról szól, és a virágok osztályai "írisz" és "rózsa". Minden virágot négy jellemzővel jellemezünk: szirmok hossza, szirmok szélessége, cseplesz hossza és cseplesz szélessége
 - Ha egy új virágot szeretnénk osztályozni, kiszámítjuk a távolságát a tanító adatkészletbeli minden virághoz. Ezután meghatározzuk a k legközelebbi virágot, és az új virág osztálycímeként a k legközelebbi virág osztálycímeként alapján határozzuk meg

Összehasonlító táblázat a gyors döntéshez

Módszer	Tanítási sebesség	Értelmezhetőség	Adatigény	Fő felhasználás
Döntési fa	Gyors	Kiváló	Alacsony/Közepes	Logikai döntések
Naiv Bayes	Nagyon gyors	Közepes	Kicsi	Szövegosztályozás
k-NN	Nincs (lusta)	Alacsony	Kicsi/Közepes	Hasonlóság alapú keresés
Neuronháló	Nagyon lassú	Nagyon rossz	Óriási	Kép/Hang, komplex minták